

PA3: LLM Fine-Tuning

Introduction

The central goal of this project was to fine-tune an encoder-only LLM to perform question answering and a decoder-only LLM to generate Python code using low-rank adaptation (LoRA) through Hugging Face. In addition to evaluating the base pretrained models, experiments focused on varying the training set size and LoRA rank to examine its capabilities as a fine-tuning technique. LoRA's background and implementation in this project are described immediately below. Sections 1 and 2 describe each task's specifications, model architectures, and datasets, as well as discussing training and evaluation results. Section 3 concludes the report with a reflection on my work to date and what I gained from it this semester.

LoRA

Low-rank adaptation is a technique for efficient fine-tuning of large machine learning models, designed to reduce the number of trainable parameters that can normally number in the millions to billions and require vast amounts of time and computational resources¹. When using LoRA, the pre-trained model weights are frozen, with updates being made via the training and composition of two lower-rank matrices. Given a weight matrix of size $d \times d$ and rank r , the decomposed adapter matrices will be of sizes $d \times r$ and $r \times d$, the sum of which results in a much smaller number of trainable parameters than the original $d \times d$. This makes training much more efficient in terms of both time and memory and, despite training far fewer parameters, still produces comparable evaluation performance to traditional fine-tuning. For this project, LoRA was implemented in both tasks via the [PEFT](#) library provided by Hugging Face.

Each task required three different kinds of experiments:

- Pre-trained model vs. fine-tuned model
- Varied training size (30%, 50%, 100%)
- LoRA rank

To compare the pre-trained and fine-tuned models, all hyperparameters were kept the same with the exception of introducing LoRA. The LoRA model used in this experiment was also used as the point of reference for the 100% training size and as one of the three values tested in the rank experiment. Regardless of rank, all LoRA models used an alpha value of $2r$ and a dropout value of 0.1.

1 Task 1: Question Answering

The first part of this project involved fine-tuning RoBERTa on the SQuAD-v2 dataset to perform a question-answering task, in which the model is given a question and context as input and outputs an answer to the question. As an encoder-only model based on BERT, RoBERTa can perform *extractive* question-answering, wherein the answer is derived from the given context rather than being newly generated. To evaluate the model's success, the reference and predicted answers were compared by Exact Match, with F1-score calculated as an additional measurement of quality.

1.1 Methods

1.1.1 Dataset & Preprocessing

The Stanford Question Answering Dataset (SQuAD)² version 2.0³ consists of over 140,000 English question-context-answer samples, originally sourced from human crowdworkers reviewing Wikipedia ar-

ticles. Each question in the dataset either has an answer located in the given context or has no answer, requiring the model to not only provide the correct answer when possible, but also to *not* answer when there is no correct answer that can be derived. Approximately 50,000 of the questions in the dataset are unanswerable.

As the Hugging Face dataset only has train and validation splits, the data was first recombined and re-split into train, dev, and test sets in an 80/10/10 ratio. The dataset then required extensive preprocessing to fit the format expected by the question-answering model. The ‘question’ and ‘context’ fields of the dataset examples were tokenized and the contexts truncated, with overlapping ID lists and an offset mapping allowing token IDs to be correctly mapped back to the original text. The ‘answers’ field was then processed to find the start and end indices of each answer in the corresponding context; these start and end positions are essentially the “labels” used by the model. For testing on the eval and test sets, those splits had a slightly different series of preprocessing steps applied, using the ‘id’ field and offset mapping to be able to verify the location of the best-scoring answer(s) in the context. These alternate splits were then used in postprocessing the output predictions, mapping the model’s guesses back to indices in the original context.

1.1.2 Model Architecture & Configurations

RoBERTa is a Transformer model based on the BERT architecture, pre-trained on approximately 160 GB of text from the BookCorpus, English Wikipedia, CC-News, OpenWebText, and Stories datasets⁴. The version of RoBERTa used in this project has over 124 million parameters. The model was instantiated using the `AutoModelForQuestionAnswering.from_pretrained()` method and trained using the standard Trainer from the transformers library.

For comparison against the pre-trained model, my baseline LoRA model had a rank of 8. The other rank values tested in the third experiment were 16 and 32. I used the following hyperparameters for all models:

- Epochs: 3
- Learning rate: 2e-5
- Batch size: 32
- Weight decay: 0.01

1.2 Results & Analysis

Training and validation loss curves for each model are displayed in Figure 1 below.

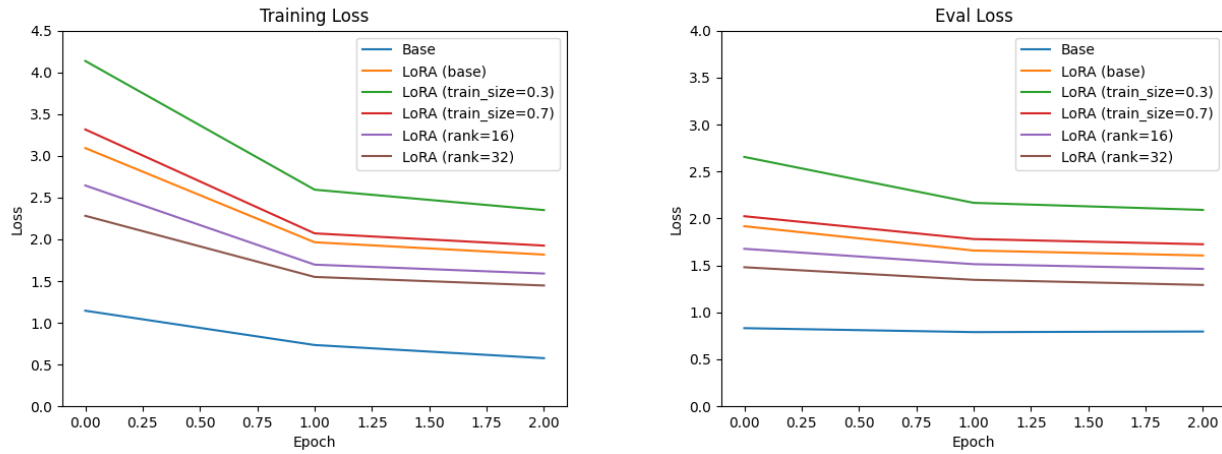


Figure 1: Training and validation loss curves for each QA model.

Exact match and F1 scores for each model are displayed in Figure 2 below.

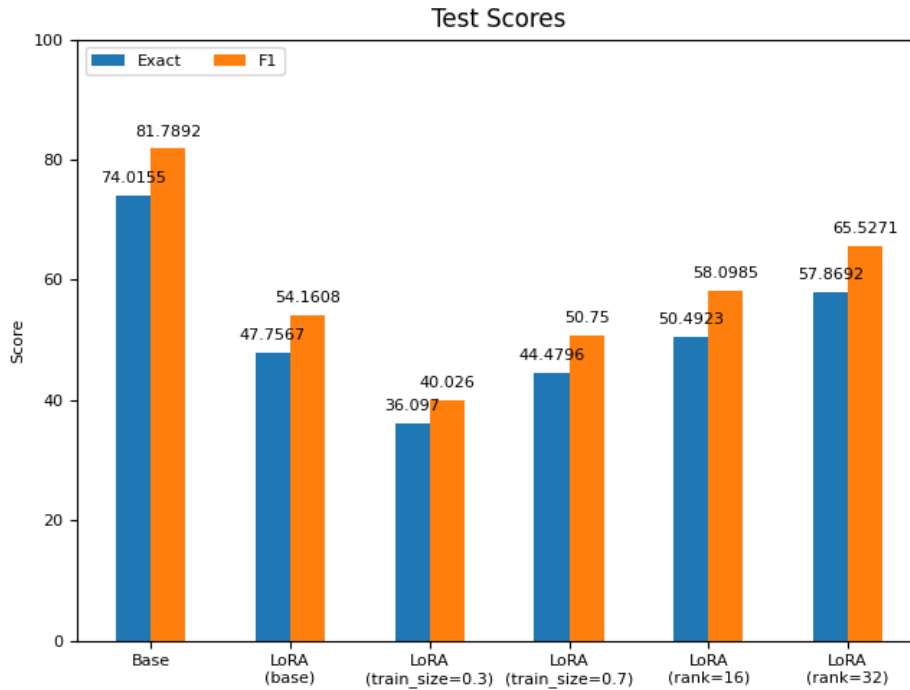


Figure 2: Exact match and F1 scores for each QA model.

Overall, training and evaluation losses and scores on the test set followed a fairly predictable pattern, with the pre-trained base model having the lowest loss and the highest test set scores, followed by the LoRA models by descending rank and then descending train set size. Given that the LoRA models have fewer trainable parameters, it makes sense that they would not perform as well as the base model when trained for the same (small) number of epochs. However, they could feasibly have been trained for more

epochs or with other adjustments like larger batch sizes due to their reduced demand for computing resources. As such, it is plausible to believe that the LoRA models are capable of achieving comparable performance. The possibility that the LoRA models would benefit from adjustments to the learning rate or other hyperparameters also warrants further study.

Further details on training and testing results are presented in Tables 1 and 2 below.

Model	Trainable Params.	Train Time (sec.)
Base	124,056,578	7098.14
LoRA (base)	296,450	2697.45
LoRA (train_size=0.3)	296,450	996.98
LoRA (train_size=0.5)	296,450	1441.64
LoRA (rank=16)	591,362	2784.23
LoRA (rank=32)	1,181,186	5598.59

Table 1: Trainable parameter count and training runtime for each model.

Model	Eval Time (sec.)	Eval Ans. Exact	Eval No-Ans. Exact
Base	181.11	68.15	85.03
LoRA (base)	91.65	43.55	55.65
LoRA (train_size=0.3)	92.69	17.17	71.62
LoRA (train_size=0.5)	90.48	40.24	52.44
LoRA (rank=16)	90.25	53.17	45.46
LoRA (rank=32)	182.89	54.96	63.32

Table 2: Testing runtime and Exact Match scores for answerable and non-answerable test set questions for each model.

Table 1 helps to visualize the reduction in parameter count from using LoRA and the relative impact it has upon training time. Unsurprisingly, reducing the training set size also reduced the training time by approximately the same percentage. Notably, however, while the rank-32 LoRA model took about twice as long to train as the rank-16 model, the rank-32 model took almost the same amount of time as the rank-8 model despite having twice as many trainable parameters. It is also interesting that the rank-32 model is closer to the pre-trained base model in terms of training time than it is to the other LoRA models, considering that it has twice as many parameters as the rank-16 model but more than 100 times fewer than the base model.

Table 2 displays a more in-depth breakdown of results on the test set as computed by the SQuAD-v2 metrics. There is another unusual divergence from the expected pattern in the running times, where the rank-8 models and the rank-16 model have approximately the same running time, but the rank-32 and base models share a running time of approximately twice that of the smaller models. The Exact Match scores for answerable questions only follow the expected trend shown by the losses and overall scores, with the LoRA model using 30% of the training set having the worst score and the pre-trained model having the best score. However, the scores for non-answerable questions (i.e. how many unanswerable questions correctly remained unanswered) have particularly atypical results. While the base model still performs the best, the second-best model is actually the LoRA model using 30% of the training set, and the worst-performing model is the rank-16 model. The rank-16 model is also the only model with a better EM score on answerable questions than non-answerable ones. Given that these EM scores do not follow the expected pattern, I question if distinguishing between answerable and unanswerable questions is a

less “deterministic” skill and thus easier for models to “accidentally” find the right weights for during short training periods, or if the training set size and possibly other hyperparameters do factor into model performance.

2 Task 2: Python Code Generation

The second part of this project involved fine-tuning the JetBrains MLLM-4b-base model on the Flytech python-codes-25k dataset to generate functional Python code, in which the model is given an instruction as input and outputs code aimed at addressing the input request. As a decoder-only model, MLLM-4b-base performs autoregressive text generation via next-token prediction, making it well-suited for constructing new text. Such models are used extensively in code-completion applications, as per MLLM-4b-base’s design.

2.1 Methods

2.1.1 Dataset & Preprocessing

The [Flytech python-codes-25k dataset](#) consists of over 24,000 samples containing English “user” instructions paired with Python code responses. The Hugging Face dataset has no splits, so the data was first split into train, dev, and test sets; this was originally performed in a standard 80/10/10 ratio, but was later redone with a 90/5/5 ratio to facilitate more reasonable evaluation speeds.

Preprocessing the dataset was much more straightforward than the steps required for the question-answering task. Due to the nature of the causal language modeling process, no additional label mapping is necessary. However, the trainer for this task requires specific dataset formats: namely, either standard or conversational-format language modeling or prompt-completion datasets. For the purposes of training the model to generate code in response to an instruction, the ‘instruction’ and ‘output’ fields of the dataset examples were mapped to the ‘prompt’ and ‘completion’ fields of a new Dataset object (standard rather than conversational format) and subsequently tokenized using the appropriate tokenizer from the `AutoTokenizer.from_pretrained()` method.

2.1.2 Model Architecture & Configurations

MLLM-4b-base is an LLM from JetBrains based on the LLaMA architecture and designed specifically for code completion, having been pre-trained on over 4 trillion tokens in multiple different programming languages⁵. With approximately 4 billion parameters, this model is much larger than RoBERTa, making it even more resource-intensive to fine-tune. The model was instantiated using the `AutoModelForCausalLM.from_pretrained()` method, and training was implemented with `SFTTrainer` from the [TRL](#) library rather than the standard transformer Trainer.

Unfortunately, despite extensive attempts to resolve a series of out-of-memory issues, I was unable to get the model to a fully functioning state on the test set, even with the use of LoRA and other techniques to reduce memory usage that improved trainability. My attempted solutions included the following:

- Altering the train/dev/test split ratios as described above
- Reducing batch size and LoRA rank
- Utilizing 4-bit quantization via the [bitsandbytes](#) library
- Loading the model with `bfloat16`
- Capping maximum sequence length at 128
- Setting eval batch size to 1

- Enabling activation offloading
- Moving output tensors to CPU every 30 eval steps
- Using a paged AdamW 32-bit optimizer
- Utilizing [Liger Kernel](#)

3 Semester Review

All three projects I completed this semester helped deepen and broaden my technical understanding of modern NLP systems as well as the practical realities of working with them. In combination with the more in-depth mathematical explanations from the course lectures, working with advanced models like RNNs, CNNs, and Transformers at a granular level enriched my understanding of how their individual components work separately and how they combine to create a full architecture. The project experiments allowed me to learn nuanced differences between certain architectures firsthand, such as how the CNNs I worked with benefited most from learning rates that were very different from the logistic regression and MLP models I examined. I also gained significant experience with implementing custom data pre-processing, as well as understanding the purpose behind the various steps involved and becoming more comfortable reading documentation to understand what different models and tasks require.

Beyond developing a firmer grasp on the concepts behind these NLP pipelines, I gained a lot of familiarity with common resources and libraries for implementing them, such as PyTorch and Hugging Face. As someone who entered the program with very limited programming and CS experience, every semester has required me to catch up very quickly on the tools involved in NLP work, and our course projects were extremely helpful in immersing me in industry-standard practices. As such, I also became much more comfortable writing my own code and not simply filling in a handful of functions in a pre-written project, as well as managing remote workflows, which paired very smoothly with my work in CLAMS. I had no previous experience with utilities like matplotlib, so our project requirements greatly improved my ability to visualize data, figure out how to extract the information I wanted from the training and evaluation processes, and overall create a pipeline that is relatively user-friendly and clear in its output.

Perhaps most importantly, I learned that any given NLP task is often much more difficult than I anticipate it to be, and I gained a much better understanding of my own limits and what can feasibly be achieved with a certain amount of time or preparation. Especially in contrast with group projects in prior courses, the work in this course highlighted the challenges of building complex systems by oneself, as well as the many different ways in which these problems can be approached and solutions implemented. To that end, our projects expanded not only the skills available to me, but also my ability to apply them in new ways and think differently about complicated tasks.

Thank you both so much for a wonderful semester!

References

- ¹ Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- ² Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In Jian Su, Kevin Duh, and Xavier Carreras, editors, *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas, November 2016. Association for Computational Linguistics.
- ³ Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don’t know: Unanswerable questions for SQuAD. In Iryna Gurevych and Yusuke Miyao, editors, *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789, Melbourne, Australia, July 2018. Association for Computational Linguistics.
- ⁴ Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692, 2019.
- ⁵ Nikita Pavlichenko, Iurii Nazarov, Ivan Dolgov, Ekaterina Garanina, Karol Lasocki, Julia Reshetnikova, Sergei Boitsov, Ivan Bondyrev, Dariia Karaeva, Maksim Sheptyakov, Dmitry Ustalov, Artem Mukhin, Semyon Proshev, Nikita Abramov, Olga Kolomyttseva, Kseniia Lysaniuk, Ilia Zavidnyi, Anton Semnenkin, Vladislav Tankov, and Uladzislau Sazanovich. Mellum-4b-base, 2025.